Textbook Formatting Rulebook

Version: 1.0
Applies to: All generated textbooks — screen and print
Status: Canonical reference — all nodes, renderers, and quality checkers defer to this document

---

How to Read This Document

Every rule has a tag:

- [CSS] — enforced mechanically in base.css. Does not require LLM cooperation.
- [PROMPT] — baked into base_prompt.py. The content generator must follow this.
- [QC] — validated by the quality checker node. Pipeline will not deliver until this passes.
- [RENDERER] — enforced in html_renderer.py at assembly time.

A rule tagged [CSS] + [QC] means: CSS tries to enforce it, and the quality checker also validates it as a backup. Belt and suspenders.

Part 1 — Typography

1.1 Font Stack

Role	Font	Fallback
Display / Headings	IBM Plex Mono	Courier New, monospace
Body prose	Source Serif 4	Georgia, serif
Code	JetBrains Mono	Courier New, monospace
Labels / Tags	DM Sans	system-ui, sans-serif

Rules:

- Never use more than these 4 typefaces in a single document. [RENDERER]
- Display font (IBM Plex Mono) is for headings, section numbers, labels only — never for body prose. [CSS]
- Body font (Source Serif 4) is for all running prose — never for headings or labels. [CSS]

- Code font (JetBrains Mono) is for all code, inline code, terminal output, and math variables referenced inline. [CSS]

1.2 Font Sizes

Screen:

Element	Size	Weight
H1 (document title)	48px	600
H2 (section title)	32px	600
H3 (subsection)	22px	500
H4 (block label)	12px	500, uppercase, letter-spacing 2px
Body prose	16px	400
Code	13px	400
Captions	13px	400, italic
Labels / tags	10px	500, uppercase, letter-spacing 1.5px
Footnotes	13px	400

Print (`@media print`):

Element	Size
H1	28pt
H2	20pt
H3	14pt
Body prose	11pt
Code	9pt
Captions	9pt

Rules:

- Never jump more than 4pt between adjacent heading levels in print. [QC]

- Captions must always be visually subordinate to body — smaller size AND colour. [CSS]
- Never go below 9pt for any printed text. [QC]

1.3 Line Height and Spacing

Element	Line Height
Body prose	1.75
Headings	1.1
Code blocks	1.6
Math display blocks	2.0
Captions	1.5

Vertical rhythm rules:

- Space above H2: 72px screen / 24pt print. [CSS]
- Space below H2: 16px screen / 8pt print. [CSS]
- Space between paragraphs: 14px screen / 8pt print. [CSS]
- Space above/below a display math block: 24px screen / 12pt print. [CSS]
- Space above/below a callout box: 20px screen / 10pt print. [CSS]
- Never let two block elements touch with zero margin between them. [QC]

1.4 Line Length

- Body prose: 60–75 characters per line. Enforced via `max-width: 720px` on the content column. [CSS]
- Absolute maximum: 90 characters. If content column exceeds this at any viewport, it is a bug. [QC]
- Code blocks: 80 characters before horizontal scroll. Lines longer than 80 chars get `overflow-x: auto`. [CSS]
- Math display blocks: centred within content column, never wider than content column. [CSS]

1.5 Text Alignment

- All body prose: left-aligned. Never justified. [CSS]
- Headings: left-aligned. [CSS]
- Math display blocks: centred. [CSS]

- Table cell numbers: right-aligned. [CSS]
- Table cell text: left-aligned. [CSS]
- Captions: centred below their figure. [CSS]

Why no justified text: Justified alignment creates uneven word spacing that slows reading, especially on narrow columns and for learners with dyslexia. Left-aligned text has consistent spacing. This is not a style preference — it is a readability requirement.

1.6 Forbidden Typography

- [CSS] No underline on anything except hyperlinks
- [CSS] No text-shadow on body copy
- [CSS] No all-caps on body prose (labels and tags are exempt)
- [CSS] No italic for emphasis in STEM content — italic is reserved for variable names and formal definitions
- [PROMPT] No bold used more than once per paragraph — if everything is emphasised, nothing is
- [PROMPT] No exclamation marks in body prose
- [QC] No consecutive bold runs — if two adjacent phrases are bold, they should be one

---

Part 2 — Colour System

2.1 The Dual-Context Principle

Every colour decision must pass contrast requirements against both the dark screen background AND white (▢ #ffffff).

Print always outputs on white paper. A colour that is legible on a dark screen but fails on white cannot be used for any text, border, or meaningful visual element.

2.2 Semantic Colour Roles

Each role has a screen value and a print-safe value. Print values activate via @media print.

Role	Screen	Print	Usage
Primary accent	#00c863	#006830	Definitions, key terms, section markers
Warning / Pitfall	#ff6b35	#b84800	Pitfall boxes, common errors
Info / Example	#4dabf7	#004f9a	Worked examples, interview anchors
Secondary	#b197fc	#5a2d8a	Connections, think boxes, prerequisites
Review / Highlight	#ffd43b	#8a6800	Review strips, star markers
Danger	#ff4757	#9a0010	Critical warnings only
Body text	#e8e8e8	#111111	All running prose
Muted text	#888888	#555555	Captions, secondary labels
Dim text	#444444	#777777	Tertiary labels, decorative
Surface	#141414	#ffffff	Page/card background
Surface raised	#1a1a1a	#f8f8f8	Callout box backgrounds
Border	#2a2a2a	#dddddd	All dividers and box borders

2.3 Contrast Requirements

Minimum contrast ratios (WCAG AA):

Pairing	Minimum Ratio
Body text on background	4.5 : 1
Large text (>18pt) on background	3.0 : 1
UI labels on background	4.5 : 1
Any text on coloured callout background	4.5 : 1

All print accent colours above have been pre-validated against #ffffff.
All screen accent colours have been pre-validated against #141414.

Do not modify these values without re-running contrast validation.

2.4 Forbidden Colour Combinations

Screen and print:

- Red text on green background — colourblind failure
- Green text on red background — colourblind failure
- Yellow text on white background — fails contrast
- Light grey text on white background — fails contrast
- Saturated colour text on saturated colour background — always fails

Print-specific (additional):

- Any text lighter than 60% grey (#666666) — disappears on paper
- Yellow, light green, light cyan, light orange as text — all wash out
- White or near-white text — does not exist on printed paper
- Dark screen backgrounds bleeding into print — must be overridden

2.5 Callout Box Colour Rules

Callout boxes use background tints on screen and border-only on print.

Screen: light tint of accent colour as background + solid left border in accent colour
Print: white or near-white background + solid left border in print-safe accent colour

```
Screen hook box:    background #0a1a0f, border-left #00c863
Print hook box:     background #f5fff8, border-left #006830

Screen pitfall box: background #120a08, border-left #ff6b35
Print pitfall box:  background #fff8f5, border-left #b84800

Screen def box:     background #141414, border-left #b197fc
Print def box:      background #f9f7ff, border-left #5a2d8a
```

Rule: [CSS] Never use a dark background on a callout box without a @media print override that switches it to a light tint.

2.6 Colour Meaning Consistency

[PROMPT] + [QC]

Once a colour is used to mean something in a document, it must mean that thing throughout. The quality checker validates:

- Green is only used for definitions and primary concepts
- Orange is only used for warnings and pitfalls
- Blue is only used for examples and interview content
- Purple is only used for connections and prerequisites
- Yellow is only used for review content

Mixing these roles within a document is a quality failure.

Part 3 — Page Layout and Margins

3.1 Content Column

Context	Value
Screen max content width	720px
Screen min side padding	48px (mobile), 80px (desktop)
Print content width	Page width minus 2 × margin
Print margin — top	1.25 inch
Print margin — bottom	1.25 inch
Print margin — outer edge	1.0 inch
Print margin — binding edge	1.5 inch (if double-sided), 1.0 inch (if single)

[CSS] Content must never touch the edge of its container. Internal padding on all block elements minimum 16px screen / 8pt print.

[QC] Any element wider than the content column is a layout error.

3.2 Sidebar Navigation (Screen Only)

- Fixed width: 230px
- Sticky, full viewport height
- Hidden on print via `@media print { #sidebar { display: none; } }`
- Hidden on mobile (below 900px) via media query
- [CSS] Sidebar never overlaps content column

3.3 Z-Index and Layering

- Sidebar: z-index 100
- Sticky headers: z-index 50
- Callout boxes: z-index 1 (normal flow)
- No element should cover readable content at any viewport width

---

Part 4 — Page Break Rules (Critical for Print)

This is the most commonly violated category in generated documents. These rules exist because breaking a table or explanation block across pages destroys comprehension — the reader must hold partial information in working memory while turning the page.

4.1 Hard Rules — Never Violate

[CSS] + [QC]

Element	Rule
Table	Never break across pages. Insert page break *before* the table if it does not fit.
Figure + caption	Must always be on the same page. Never separate.
Worked example block	Never split. Start on new page if it does not fit on remaining space.
Definition box	Never split. Start on new page if needed.
Callout box (any type)	Never split.
Code block	Never split mid-block. Prefer page break before over splitting.
Section heading	Must be followed by at least 3 lines of content on the same page. Never stranded at bottom.
Diagram	Must stay with its caption and ideally with the paragraph that introduces it.

4.2 Soft Rules — Apply Where Possible

[CSS]

- Minimum 3 lines of a paragraph kept together — no orphan single lines at top or bottom of page
- Break paragraphs at sentence boundaries only, never mid-sentence
- Keep a section's hook paragraph and first major block on the same page where possible

- Prefer more whitespace at bottom of a page over breaking a block — generous bottom margin is better than a split element

4.3 CSS Implementation

```
/* ── Hard breaks — never split ──────────────────────────────── */
table,
figure,
.worked,
.def-box,
.hook,
.pitfall,
.interview,
.think,
.review,
.code-wrap,
.diagram {
  break-inside: avoid;
  page-break-inside: avoid; /* legacy browsers */
}

/* ── Headings — keep with following content ──────────────────── */
h1, h2, h3, h4 {
  break-after: avoid;
  page-break-after: avoid;
}

/* ── Orphan and widow control ────────────────────────────────── */
p {
  orphans: 3;
  widows: 3;
}

/* ── Force page break before large elements ──────────────────── */
/* Applied by renderer when element height > remaining page estimate */
.force-page-break-before {
  break-before: page;
  page-break-before: always;
}
```

4.4 PDF/WeasyPrint Implementation

When generating PDF via WeasyPrint, the CSS above applies directly. For ReportLab:

```python
from reportlab.platypus import KeepTogether, PageBreak

# Wrap any block that must not split
block = KeepTogether([
    table_flowable,
])

# Headings always followed by KeepWithNext
heading = Paragraph("Section Title", heading_style)
heading.keepWithNext = True

# Definition boxes
def_block = KeepTogether([
    def_label,
    def_term,
    def_body,
])
```

Part 5 — Math Notation

5.1 Display vs Inline

[PROMPT]

Use inline math when	Use display math when
Referencing a variable mid-sentence: "where n is the input size"	The equation is the point of the sentence
Short expressions: O(n log n)	More than two operations
Variable names: x, y, θ	The reader needs to study it, not glance at it
	It is referenced later by equation number

Rule: If in doubt, use display math. Inline math that is too complex slows reading more than a display block does.

5.2 Spacing Rules

[PROMPT] + [QC]

- Binary operators always spaced on both sides: a + b not a+b
- Assignment and equality spaced: x = y not x=y
- Fractions rendered as proper fractions — never forward-slash inline for display math
- Vertical breathing room above and below every display block: 24px screen / 12pt print
- Summation and integral signs: never inline with dense prose

5.3 Notation Consistency

[PROMPT] + [QC]

- Same concept gets the same symbol throughout the document
- Vectors: always bold lowercase **v** or arrow notation — pick one, enforce it
- Matrices: always bold uppercase **A** — never switch
- Sets: always calligraphic or double-struck — never plain uppercase that could be confused with a matrix
- Every symbol used must be defined the first time it appears
- No symbol redefinition — once n means input size, it means input size everywhere

5.4 Equation Numbering

[RENDERER]

- Every equation that is referenced later gets a right-margin number: (1), (2), (3)
- Numbering is sequential through the whole document, not per section
- References in prose: "as shown in (3)" not "as shown above"

5.5 Subscripts and Superscripts

[PROMPT] + [QC]

- Always use proper markup — never Unicode substitutes ($_{0123}$, ^{0123})
- Reason: Unicode sub/superscripts render as black boxes in many PDF renderers and are inaccessible to screen readers
- In HTML: use MathJax or <sub> / <sup> tags
- In PDF: use ReportLab's <sub> / <super> XML tags in Paragraph objects

---

Part 6 — Tables

6.1 Structure Rules

[PROMPT] + [QC]

- Every table has a title above it
- Column headers always present — no headerless tables
- No merged cells in the first column — it breaks screen reader order
- Maximum 6 columns before the table should be split or restructured
- If a table has more than 8 rows, use alternating row shading

6.2 Alignment

[CSS]

- Numbers: right-aligned
- Text: left-aligned
- Headers: match their column content alignment
- Units in headers, not in cells: "Speed (m/s)" not repeating "m/s" in every cell

6.3 Padding

[CSS]

- Minimum cell padding: 10px vertical, 12px horizontal (screen)
- Print: 6pt vertical, 8pt horizontal
- Never let cell content touch cell border

6.4 Page Break

[CSS] + [QC]

- Tables never break across pages — this is a hard rule from Part 4
- If a table is too tall to fit on one page, the renderer must either:
  - Insert a page break before the table, or
  - Split the table at a logical row boundary with a "continued" header on the next page
- The quality checker flags any table estimated taller than 60% of a print page

6.5 Print Colour

[CSS]

- Header row: light grey background #f0f0f0 on print (not the screen accent colour)
- Alternating rows: #f8f8f8 and #ffffff
- Borders: #cccccc — visible but not heavy

Part 7 — Code Blocks

7.1 Structure

[RENDERER]

Every code block has:

- A header bar with: decorative dots (macOS style), language label right-aligned
- Syntax highlighting via Prism.js (screen) or equivalent
- Line numbers for blocks longer than 10 lines
- A label/badge indicating what the block demonstrates (e.g. "Pattern Template", "Worked Example")

7.2 Line Length

[CSS] + [QC]

- Maximum 80 characters per line
- Lines longer than 80 characters: `overflow-x: auto` on the block — horizontal scroll, never wrap
- Wrapping code is harder to read than scrolling code — never wrap
- The quality checker flags code blocks with lines exceeding 80 characters and no overflow handling

7.3 Print Rules

[CSS]

```
css
```

```
@media print {
  .code-wrap {
    background: #0d0d0d !important; /* Keep dark — code on dark is legible in print */
    color: #e8e8e8 !important;
    border: 1px solid #cccccc;
    font-size: 9pt;
    line-height: 1.5;
    white-space: pre;          /* Never wrap */
    overflow: visible;         /* Let it overflow rather than cut off */
    word-break: normal;
  }

  /* If a code block is too wide, scale it down rather than wrap */
  .code-wrap pre {
    max-width: 100%;
    transform-origin: top left;
  }
}
```

Note: Code blocks are one of the few elements that keep a dark background on print. Black-on-white code lacks the visual separation that makes it scannable. Dark code blocks on a white page are acceptable and conventional in technical textbooks.

7.4 Comments

[PROMPT]

- Every non-trivial code block has inline comments explaining the *why*, not the *what*
- Comment density: at least one comment per logical block (3–5 lines)
- Comments in body text colour, not accent colour
- No commented-out code in generated examples

Part 8 — Diagrams and Figures

8.1 SVG Rules

[RENDERER] + [QC]

- All SVGs are inline — no external `src` references
- Every SVG has explicit `width`, `height`, and `viewBox` attributes
- SVG background must match page background — `fill="#141414"` screen, `fill="#ffffff"` print

- All SVG text uses the code font (IBM Plex Mono) for labels
- Minimum SVG text size: 9px screen / 7pt print — smaller is not readable
- Arrow markers defined inline in <defs> — no external marker references

8.2 Print SVG Override

[CSS]

```
@media print {
  svg {
    background: #ffffff !important;
  }
  svg rect[fill="#141414"],
  svg rect[fill="#1a1a1a"],
  svg rect[fill="#0c0c0c"] {
    fill: #ffffff !important;
  }
  svg text {
    fill: #111111 !important; /* Dark text on white print background */
  }
}
```

Problem this solves: SVGs generated for dark-mode screen have dark backgrounds. On print (white paper), those dark rectangles print as solid black blocks, obliterating the diagram content. The CSS override forces SVG backgrounds white and text dark before printing.

8.3 Figure Organisation

[RENDERER] + [QC]

- Every figure is wrapped in a <figure> element with a <figcaption>
- Caption always below the figure, never above
- Figure and caption never separated across pages (Part 4 hard rule)
- Figures referenced in prose before they appear: "as shown in Figure 3" not "as shown below"
- Figure numbering sequential through the whole document

8.4 Diagram Sizing

[CSS]

- Maximum diagram width: content column width (720px screen)

- Minimum diagram height: 120px — smaller diagrams are not worth including
- On mobile: diagrams scale down with `max-width: 100%` — never overflow
- Print: diagrams scale to fit content column, maintaining aspect ratio

---

Part 9 — Callout Boxes

9.1 The Five Box Types

Type	Trigger word	Screen border	Print border	Usage
Hook	💡 INTUITION HOOK	#00c863	#006830	Opens every section — the felt problem
Definition	📐 DEFINITION	#b197fc	#5a2d8a	Formal definitions only
Pitfall	⚠️ PITFALL	#ff6b35	#b84800	Common errors, misconceptions
Interview	🎯 INTERVIEW ANCHOR	#4dabf7	#004f9a	Interview-specific advice
Think	🔬 THINK FIRST	#b197fc	#5a2d8a	Conjecture prompts

9.2 Rules

`[PROMPT]` + `[QC]`

- One Hook per section — always the first element after the section header
- Maximum one Pitfall per section unless the section is specifically about error types
- Never nest one callout box inside another
- Never use a callout box for content that is not genuinely that type — a definition box must contain a definition, not a general explanation
- Maximum 4 sentences in a Pitfall box — if it needs more, it is prose, not a pitfall

9.3 Print Implementation

`[CSS]`

```
css
```

```
@media print {
  .hook {
    background: #f5fff8 !important;
    border-left: 3px solid #006830 !important;
    color: #111111 !important;
  }
  .def-box {
    background: #f9f7ff !important;
    border-left: 3px solid #5a2d8a !important;
    color: #111111 !important;
  }
  .pitfall {
    background: #fff8f5 !important;
    border-left: 3px solid #b84800 !important;
    color: #111111 !important;
  }
  .interview {
    background: #f5f8ff !important;
    border-left: 3px solid #004f9a !important;
    color: #111111 !important;
  }
  .think {
    background: #f9f7ff !important;
    border-left: 3px solid #5a2d8a !important;
    color: #111111 !important;
  }

  /* Remove all dark backgrounds from box internals */
  .hook *, .def-box *, .pitfall *,
  .interview *, .think * {
    color: #111111 !important;
    background: transparent !important;
  }
}
```

Part 10 — Content Organisation

10.1 Section Structure Is Fixed

[PROMPT] + [QC]

Every section follows this exact order. No deviations.

1. Section header (number + title + subtitle)
2. Intuition Hook box
3. Prerequisites / Connects-to block (if any)
4. Body prose — plain explanation
5. Definition box (if section introduces a formal concept)
6. Diagram (if needed)
7. Worked example (if needed)
8. Code example (if needed)
9. Pitfall box (if there is a genuine common error)
10. Practice problems (3 per section: warm / medium / cold)
11. Interview anchor box (if applicable)

The reader who has read section 3 knows exactly what to expect in section 7. This predictability reduces cognitive load — the learner spends mental energy on the content, not on navigating unfamiliar structure.

10.2 Heading Hierarchy

[PROMPT] + [QC]

- H1: document title only — once per document
- H2: section titles
- H3: named subsections within a section
- H4: block labels (Definition 3.1, Example 4.2 etc.)
- Never skip levels — no H4 without a H3 parent
- Never use a heading followed immediately by another heading with no content between them

10.3 Worked Examples

[PROMPT] + [RENDERER]

- Numbered sequentially through the whole document: Example 1.1, 1.2, 2.1 etc.
- Every step of a worked example explains the *why*, not just the *what*
- Last step always states the conclusion explicitly — never end mid-calculation
- Steps presented in a two-column grid: label on left, explanation on right
- The worked example block never splits across pages

10.4 Practice Problems

[PROMPT] + [RENDERER]

- Always 3 per section: Warm (accessible), Medium (requires thought), Cold (stretches)

- Each has: a difficulty badge, a problem statement, a hint, and write-in space
- Hints are genuinely helpful — not "think about the problem" but a specific nudge
- No solutions in the textbook body — solutions are a separate document or appendix
- Write-in lines for screen: `<input>` fields styled to look like ruled lines
- Write-in lines for print: actual ruled lines (CSS border-bottom on blank divs)

10.5 Intentional Whitespace

`[CSS]`

Generated documents tend to be dense. Whitespace is not wasted space — it is cognitive breathing room.

- Every H2 section has 72px of space above it on screen, 24pt in print
- After a worked example block: 32px extra space before next element
- After a callout box: 20px space
- The last element in a section before the next section header: 56px space
- Never compress spacing to fit more content on a page — break to next page instead

---

Part 11 — Quality Checker Validation Rules

The quality checker runs after assembly and before delivery. These are the checks it performs. A failed check triggers a targeted re-run of the relevant node.

11.1 Structure Checks

Check	Failure action
Every section in curriculum plan is present	Re-run content generator for missing section
Every section has a heading	Re-run content generator for that section
Section order matches curriculum plan reading order	Re-run assembler
No heading followed immediately by another heading	Re-run content generator
Worked examples are numbered sequentially	Re-run renderer
Practice problems: exactly 3 per section, one of each difficulty	Re-run content generator

11.2 Content Checks

Check	Failure action
No section references a concept not yet introduced	Re-run content generator with prerequisite context
Every symbol is defined on first use	Re-run content generator for that section
Same term used for the same concept throughout	Re-run quality pass with glossary enforcement
No symbol redefined with a different meaning	Flag for manual review
Difficulty does not spike — each section max one complexity step above prior	Re-run planner if systemic, re-run section if isolated

11.3 Formatting Checks

Check	Failure action
No table estimated taller than 60% of a print page without page-break-before class	Re-run renderer with break annotation
No code lines exceeding 80 characters without overflow handling	Re-run renderer
No inline Unicode sub/superscripts ($_{012}$, ^{012})	Re-run content generator
No empty diagram tags	Re-run diagram generator
No callout box nested inside another	Re-run content generator
No definition box with empty body	Re-run content generator
All SVGs have width, height, viewBox attributes	Re-run diagram generator

11.4 Colour and Contrast Checks

Check	Failure action
All text colours meet 4.5:1 contrast on both screen background and white	Re-run renderer with corrected colour values
No forbidden colour combinations present	Re-run renderer
All callout boxes have print overrides in CSS	Static — never changes, fixed in base.css

11.5 Code Checks

Check	Failure action
All Python code blocks pass `ast.parse()`	Re-run code generator for that section
No commented-out code in examples	Re-run code generator
All code blocks have language label	Re-run renderer
Expected output matches actual output (where verifiable)	Re-run code generator

Implementation Guide for the Coding Agent

Where Each Part Lives in the Codebase

Rulebook Part	Enforcement Location	File
Typography (Part 1)	CSS + prompt	`services/renderer/assets/base.css`, `services/prompts/base_prompt.py`
Colour system (Part 2)	CSS	`services/renderer/assets/base.css`
Page layout (Part 3)	CSS	`services/renderer/assets/base.css`
Page break rules (Part 4)	CSS + renderer + QC	`base.css`, `services/pipeline/quality_checker.py`
Math notation (Part 5)	Prompt + QC	`services/prompts/content_prompts.py`, `quality_checker.py`
Tables (Part 6)	CSS + prompt + QC	All three
Code blocks (Part 7)	CSS + renderer + prompt	All three
Diagrams (Part 8)	CSS + renderer + QC	All three
Callout boxes (Part 9)	CSS + prompt + QC	All three
Content organisation (Part 10)	Prompt + renderer + QC	All three

Rulebook Part	Enforcement Location	File
Quality checker rules (Part 11)	QC	`services/pipeline/quality_checker.py`

The `@media print` Block

The entire print override lives in one place at the bottom of `base.css`. It is never split across multiple files. Structure:

```
@media print {
  /* 1. Hide navigation elements */
  /* 2. Reset backgrounds and colours */
  /* 3. Callout box print overrides */
  /* 4. SVG print overrides */
  /* 5. Code block print overrides */
  /* 6. Page break rules */
  /* 7. Font size overrides */
  /* 8. Margin overrides */
}
```

The Base Prompt Injection

`base_prompt.py` exports one string: `BASE_PEDAGOGICAL_AND_FORMATTING_RULES`. It is injected as a system-level preamble into every content-generating node call. It covers:

- The pedagogical invariants (Feynman rules, plain English first)
- The formatting rules that only the LLM can enforce (spacing around operators, notation consistency, callout box usage rules, heading hierarchy)

The LLM is not trusted to enforce CSS-level rules — those are mechanical. The LLM is only given the formatting rules it is actually capable of following in its text output.

Priority Order for the Coding Agent

Build in this order:

1. base.css (screen styles) — before any renderer work
2. @media print block — immediately after screen styles, not later
3. base_prompt.py — before any content generator work
4. quality_checker.py structure — the scaffold, even before all checks are written
5. Content generator nodes — with base_prompt already injected
6. Renderer — base.css already exists, just assemble
7. Quality checker checks — fill in as nodes are completed

The single most common mistake when implementing a system like this is deferring the print styles until "later." There is no later — once content is generated against screen-only styles, retrofitting print compatibility is expensive. Build `@media print` on day one.

End of Formatting Rulebook v1.0